

DeepTraffic-Guard

An AI camera system that catches traffic violations automatically

A complete, plain-language guide for someone with zero background
on computer vision, AI, or this project.

Live demo: <https://deeptraffic-guard.streamlit.app>

What This Document Covers

1. The problem this project solves
2. The big idea, explained simply
3. The complete journey of one photo through the system
4. Step 1 -- Cleaning up the image
5. Step 2 -- Finding every object in the scene (AI detection)
6. Step 3 -- Working out who is riding what
7. Step 4 -- Finding the road rules in the photo itself
8. Step 5 -- Checking all eight violation types (in detail)
9. Step 6 -- Reading license plates
10. Step 7 -- Producing evidence: marked photos and PDF reports
11. The video mode
12. Batch processing -- handling many photos at once
13. The dashboard: where all violations get stored and searched
14. The technology stack, explained piece by piece
15. How we tested it for real, and the bugs that testing found
16. Honest limitations -- what it can't do (yet)
17. Glossary of terms used in this document

1. The Problem This Project Solves

Every city with traffic has the same basic problem: there are far more roads than there are police officers to watch them. A traffic officer standing at one intersection can only watch that one intersection. The moment they look away, or move to a different junction, violations at the first location go completely unnoticed.

At the same time, almost every city already has the hardware needed to fix this: traffic cameras. They are mounted on poles, bridges, and signal posts everywhere, quietly recording footage that, in most cases, nobody is actively watching in real time. The footage exists. The watching does not.

This creates a gap. Common violations -- riding without a helmet, three people crammed onto one motorcycle, driving past a red light, parking on a curb marked 'no parking' -- happen constantly, but are caught only when an officer happens to be standing right there at that exact moment. Most of the time, nobody is.

Why this matters beyond just 'catching rule-breakers'

Helmet and seatbelt laws exist because they save lives in a crash. Triple riding makes a motorcycle unstable and harder to control. Running red lights and driving on the wrong side of the road are leading causes of serious accidents. None of these rules exist to generate fines -- they exist because ignoring them gets people killed. A system that can watch 24 hours a day, without getting tired or distracted, closes the enforcement gap that currently lets so many of these violations go unaddressed.

2. The Big Idea, Explained Simply

DeepTraffic-Guard takes a single photo or a short video clip from a traffic camera and automatically does what a trained human inspector would do if they watched that same footage: it identifies every vehicle and person in the frame, works out which rules are being broken, and produces proof.

You don't need to tell the system anything about the location beforehand. You don't need to mark out where the stop line is, where the no-parking zone is, or where the lanes are. The system figures all of that out by actually looking at the photo, the same way a person would just glance at the road and immediately know where the stop line is painted.

Why 'no manual setup' is the hard part

Many simpler systems require someone to sit down beforehand and draw boxes on a map saying 'this rectangle is the no-parking zone' or 'this line is the stop line' for every single camera. That doesn't scale -- it means a human has to manually configure every new camera location. DeepTraffic-Guard instead looks for the actual painted road markings (white stop-line paint, yellow/red curb paint, road signs) directly in each photo, so the exact same system works at a brand-new intersection it has never seen before, with zero setup.

What you get back

For every photo or video you feed in, the system gives you: a marked-up image with colored boxes around every violation, a written explanation of which rule was broken and how confident the system is, the vehicle's license plate (read automatically), and -- if you want it -- a downloadable PDF report that looks professional enough to use as actual evidence.

3. The Complete Journey of One Photo Through the System

To understand the system, it helps to follow a single photo from the moment you upload it to the moment you get results. Here is the full path, at a glance -- each step is explained in much more detail in the sections that follow.

- **1. Upload**

You drag a photo (or video) into the website.

- **2. Clean up**

The system checks if the image is blurry, too dark, or has harsh shadows, and fixes whichever of those apply.

- **3. Detect objects**

An AI model scans the cleaned-up image and draws a box around every car, motorcycle, bus, truck, person, and traffic light.

- **4. Link riders to bikes**

Any person sitting on a motorcycle gets relabeled as a 'rider' of that specific bike, so later checks know who is actually riding versus who is just standing nearby.

- **5. Find road rules**

The system scans the photo itself for painted stop lines, curb paint marking no-parking zones, and road signs.

- **6. Check every rule**

Each of the 8 violation checks runs against every relevant vehicle or rider in the photo.

- **7. Read plates**

For every flagged vehicle, the system zooms into the license plate area and reads the text.

- **8. Produce evidence**

Boxes get drawn on the image, a cropped close-up of each violation is saved, and you can export everything as a PDF.

- **9. Save & search**

If you choose to save it, the violation gets logged into a database you can search and chart later from the dashboard.

4. Step 1 -- Cleaning Up the Image

Real traffic camera photos are rarely perfect. They might be slightly blurry from camera shake, taken at dusk so they're too dark, or have one side in bright sun and the other in deep shadow under a bridge. Before any analysis happens, the system checks for three specific problems and fixes only the ones that are actually present -- it never alters a photo that doesn't need it.

Problem 1: Blur

The system measures how 'sharp' the image is using a mathematical technique that looks at how abruptly brightness changes between neighboring pixels (a sharp edge changes abruptly; a blurry edge changes gradually). If the image is below a sharpness threshold, it applies a sharpening filter -- the same basic idea as the 'sharpen' tool in a photo editor.

Problem 2: Low light

The system checks the average brightness of the image. If it's too dark, it applies a technique called CLAHE (Contrast Limited Adaptive Histogram Equalization) -- in plain terms, this brightens dark areas and pulls out detail that was hiding in the shadows, without blowing out the parts of the image that were already bright.

Problem 3: Uneven shadows

Photos taken under a flyover or bridge often have one well-lit side and one shadowed side. The system detects this unevenness and corrects only the brightness of the image, carefully leaving the actual colors untouched -- this matters a lot, because several violation checks later on (like spotting a red helmet correctly) depend on colors being accurate. An earlier version of this project had a bug where this correction step accidentally washed out colors across the whole photo; it was fixed so brightness and color are now corrected independently.

5. Step 2 -- Finding Every Object in the Scene

This is the core AI step. The cleaned-up image is fed into a model called YOLOv8 (You Only Look Once, version 8) -- a type of neural network that has been trained on millions of photos to recognize common objects like people, cars, motorcycles, buses, trucks, and traffic lights.

What does 'detecting an object' actually mean?

For every object it recognizes in the photo, the model produces three things: a bounding box (the pixel coordinates of a rectangle drawn tightly around the object), a class label (what it thinks the object is, e.g. 'motorcycle'), and a confidence score (how sure it is, from 0% to 100%). The system only keeps detections above a confidence threshold (35% by default) to avoid acting on guesses that are too uncertain.

Cleaning up duplicate detections

Sometimes the model draws two overlapping boxes for the same real object (for example, mistaking the front and back of one long bus for two separate vehicles). The system checks for boxes that overlap heavily and keeps only the one with the highest confidence, discarding the duplicate.

6. Step 3 -- Working Out Who Is Riding What

The object detector finds 'person' boxes and 'motorcycle' boxes completely separately -- it has no built-in concept of one person sitting on one specific bike. This matters a lot, because the helmet, triple-riding, and phone-use checks only make sense for someone who is actually riding, not for a pedestrian who happens to be walking past a parked motorcycle.

How the system links them

For every person detected, the system measures how much their box physically overlaps with each motorcycle's box, as a percentage of the person's own box area. If a large enough fraction of the person's box -- particularly their lower body -- overlaps the motorcycle, they get relabeled from 'person' to 'rider' and linked to that specific bike.

A real bug this caught, and how it was fixed

An earlier version of this overlap check was too loose: it also counted anyone simply walking near a motorcycle, even with zero physical overlap, as long as they were roughly in the same area of the photo. This caused a walking pedestrian who happened to be near a parked bike to get incorrectly flagged for 'no helmet', since the system thought they were riding it. The fix tightened the rule so a person must have genuine, measurable overlap with the motorcycle's box -- a person standing nearby with no overlap at all is correctly left alone as just a pedestrian.

7. Step 4 -- Finding the Road Rules in the Photo Itself

This is the part of the system that avoids needing any manual setup. Instead of being told in advance where the stop line or no-parking zone is, the system looks for the actual paint on the road in every single photo it processes.

Finding the stop line

A painted stop line is a thick band of bright white paint stretching across most of the road's width. The system scans the image row by row, looking for rows where a large fraction of pixels are bright white -- a real stop line shows up as a dense, continuous band, while random white objects (like a white car) don't form that pattern across the full road width. As a second check, it also looks for a straight horizontal line at roughly that same location using a line-detection technique called the Hough Transform. If neither check finds convincing evidence, the system simply reports that no stop line was found -- it never assumes a default position.

Finding no-parking zones

No-parking zones are usually marked with yellow or red curb paint near the camera. The system searches only the bottom portion of the photo (since curb paint closest to the camera appears at the bottom of the frame), looking for color patterns matching that paint, and specifically rejects short paint smudges or color blobs that don't form a long, thin, continuous strip the way real curb paint does.

Finding road signs

The system looks for sign-shaped, sign-colored regions (red for stop/no-entry style signs, blue for informational signs) in the upper portion of the image, where signs are physically mounted -- and filters out shapes that are too large, too small, or the wrong proportions to plausibly be a sign (this avoids mistaking billboards or building facades for road signs).

8. Step 5 -- Checking All Eight Violation Types

With every object identified, every rider linked to their bike, and every road marking located, the system now runs eight independent checks. Each check only looks at the specific vehicles or people it's relevant to, and produces a confidence score for how sure it is a violation actually occurred.

1. Helmet non-compliance

For every rider, the system crops out just the top portion of their body (the head region) and looks for three independent clues that a helmet is missing: visible skin tone (a bare face), visible dark hair texture (the top of someone's head, viewed from behind or above, looks rough and irregular -- very different from a smooth helmet shell), and overall surface texture (hair has high visual 'noise'; a helmet's surface is smooth). A special check also exists to avoid a tricky false alarm: brightly colored helmets (red, orange, yellow) have shiny highlights that can look like skin tone to a naive color check, so the system separately checks whether a large portion of the head region is covered in vivid, saturated paint-like color -- if so, it treats that as strong evidence of an actual helmet shell and backs off the skin-tone signal.

2. Triple riding

Simply counts how many people have been linked as riders to the same motorcycle. More than two riders on one bike (the legal maximum in most places) triggers this violation.

3. Stop-line violation

Checks whether any vehicle's furthest-forward edge has crossed past the auto-detected stop line. This check is automatically skipped entirely if no stop line was found in that particular photo -- the system never guesses where one 'probably' is.

4. Red-light violation

Builds on the stop-line check, but adds one more condition: it crops the detected traffic light and checks whether the upper portion (where the red light sits) shows a strong red color. Only if the light is confirmed red AND a vehicle has crossed the stop line does this fire -- crossing the line while the light is green or yellow is not flagged.

5. Wrong-side driving

Looks for lane markings using line-detection on the lower half of the photo, estimates where the center of the road should be, and flags vehicles whose position and orientation suggest they're driving on the wrong half of a simple two-lane road. This check works best on straightforward two-lane roads and is intentionally conservative on complex multi-lane highways, where the idea of a single 'lane center' doesn't really apply.

6. Illegal parking

Checks how much of a stationary vehicle's box overlaps with the auto-detected no-parking curb-paint zone. Like the stop-line check, this is skipped entirely if no parking zone was found in the photo.

7. Seatbelt non-compliance

For people detected inside a car, the system crops the chest/torso area and looks for a diagonal line pattern -- a real seatbelt strap creates a consistent diagonal edge across the chest. If that diagonal pattern is missing, it's flagged,

though with a deliberately capped confidence score since this particular check is the least certain of the eight (clothing patterns can sometimes create similar diagonal lines).

8. Phone use while driving or riding

The object detector also recognizes 'cell phone' as a class. If a detected phone overlaps with a rider's hands, or with a person who is themselves sitting inside a car, this fires -- catching both motorcycle riders and car drivers using their phone.

9. Step 6 -- Reading License Plates

For every car, motorcycle, bus, or truck the system finds, it crops out the lower portion of the vehicle's box (where the plate is usually mounted), cleans up that small crop (removing noise, sharpening edges, boosting contrast between the plate's background and its characters), and then runs it through EasyOCR -- a ready-made text-recognition tool trained specifically to read characters in photos.

The recognized text is then checked against the typical pattern of a license plate (a specific sequence of letters and numbers). If the OCR result doesn't match a believable plate pattern, or the plate is too blurry to read at all, the system honestly reports 'unreadable' rather than guessing at characters it isn't confident about.

10. Step 7 -- Producing Evidence: Marked Photos & PDF Reports

Detecting a violation isn't useful unless you can actually see and communicate it. The final stage of the pipeline turns raw detections into something a person can immediately understand and act on.

The annotated image

The system draws colored boxes around every violation (color depends on how serious it is -- red for critical, orange for high, yellow for medium, blue for low), with a label naming the violation and a confidence percentage. Any auto-detected stop line, parking zone, and road signs are also drawn in, so you can visually confirm what the system 'saw' and why it made each decision. A small crop of each individual violation is also saved separately as a close-up evidence image.

The PDF report

With one click, the system can generate a formatted PDF: a header with the camera location and timestamp, the full annotated photo, a clean table listing every violation with its severity, confidence, and license plate, and a dedicated page for each violation's close-up evidence photo. This is designed to look like something you could actually hand to someone as documentation, not just a raw data dump.

11. The Video Mode

Everything described so far works on a single photo. The system also has a video mode, since most real traffic cameras record continuous video rather than single snapshots.

Processing every single frame of a video at full detail would be very slow, so the system samples frames at a configurable interval (for example, every 5th frame) -- it runs the full detection-and-violation pipeline on those sampled frames, while still writing every frame to the output video so the result plays back smoothly.

After processing, you get: a downloadable annotated video with every detected violation marked, a summary count of how many times each violation type occurred across the whole clip, thumbnail images of the key moments where violations were caught, and the same one-click PDF report, summarizing the entire video instead of a single photo.

12. Batch Processing -- Handling Many Photos at Once

Everything described so far processes one photo (or one video) at a time. In the real world, a system like this would need to handle footage from many cameras across a city, not just one upload at a time -- so the same Analyze Image screen also accepts multiple photos in a single upload.

When more than one photo is uploaded together, the system switches into a batch mode: it runs the full pipeline (clean up, detect, find zones, check violations, generate evidence) on every photo one after another, then shows you a combined summary instead of just one result.

What batch mode shows you

Total images processed, total objects found across all of them, total violations found, and -- importantly -- the actual throughput: how many images per second the system managed, based on real measured time, not an estimate. Each individual photo's result is still available to expand and view in detail, and you can download every annotated photo at once as a single ZIP file, or save every violation found across the whole batch to the database in one click.

Why this matters

A system that can only handle one photo at a time isn't something a city could actually deploy across many cameras -- it would need a person sitting there uploading one photo, waiting, uploading the next. Batch mode is a direct, working demonstration that the same pipeline scales to handle a fleet of camera feeds, not just a single example.

13. The Dashboard: Storing & Searching Violations

Every violation you choose to save gets written to a small built-in database (SQLite -- a lightweight, file-based database that doesn't need a separate server to run). The system also computes a unique fingerprint for each photo so the exact same frame can't accidentally be saved twice.

From the Dashboard tab, you can see: the total number of violations recorded, how many happened today, a breakdown by violation type and by severity, a chart of violations by hour of day (useful for spotting patterns, like more violations during evening rush hour), and a searchable, filterable table of every individual record -- searchable by date range, violation type, severity, or even a specific license plate. The whole table can also be exported as a CSV file for use in spreadsheets or other reporting tools.

14. The Technology Stack, Explained Piece by Piece

- **YOLOv8 (Ultralytics)**

The AI model that finds and labels objects in images. 'You Only Look Once' refers to its design: instead of scanning an image many times in different ways, it looks at the whole photo once and predicts every object's location and label simultaneously, which is part of why it runs fast enough for near-instant results.

- **OpenCV**

A widely used open-source toolkit for classic (non-AI) image processing: color analysis, edge and line detection, blurring, sharpening, and shape analysis. Most of the violation checks (helmet color analysis, stop-line detection, seatbelt diagonal-line detection) are built directly on OpenCV tools rather than another AI model, since these are well-defined visual patterns that classic image processing handles reliably.

- **EasyOCR**

A ready-made text-recognition library used specifically to read license plate characters from the cropped plate region.

- **Streamlit**

The framework that turns this Python project into an actual interactive website, without needing to separately build a traditional web front-end. It handles the file upload, buttons, sliders, image display, and charts you see in the browser.

- **SQLite**

A simple, file-based database used to permanently store every saved violation so it can be searched and charted later from the dashboard.

- **fpdf2**

A Python library used to generate the downloadable PDF evidence reports directly from the violation data and images, with no manual formatting required each time.

15. How We Tested It For Real, And The Bugs That Testing Found

It's easy to demo a system on two or three hand-picked photos and call it done. That doesn't tell you much about whether it actually works. Instead, this project was tested the way a real evaluation works: gather real photos, write down -- before looking at the system's answer -- what the correct answer should be, then run the system and honestly compare the two.

Three numbers that matter, in plain terms

Precision answers: 'Of everything the system flagged as a violation, how much of it was actually a real violation?' A low precision means too many false alarms. Recall answers the opposite question: 'Of all the real violations that existed, how many did the system actually catch?' A low recall means it's missing real problems. F1-score is a single number that balances both, so a system can't game one at the expense of the other.

On a hand-checked set of real photos, helmet detection reached 100% recall (it never missed a real violation in this test set) with 60% precision (some false alarms remain, fully explained below) -- an overall accuracy of 93.75% across every violation type that had enough test examples to measure honestly.

A second, different measurement: how good is the AI model itself?

The numbers above measure whether a violation was correctly flagged. A separate, stricter measurement called mAP (mean Average Precision) checks something more basic: did the underlying AI model draw its boxes in the right place at all, regardless of violations? This required manually drawing the 'correct' box around every car, person, and motorcycle in several real photos by eye, then checking how well the system's own boxes lined up with them. The result: an mAP of 0.869 -- on a scale where 1.0 would mean every single box was placed perfectly.

Real bugs this testing found and fixed

This wasn't a clean pass. Testing against real, messy photos surfaced several genuine bugs -- each one is a useful story in its own right:

- **The red-helmet false alarm**

A rider wearing a bright red helmet was flagged as having no helmet at all. The reason: pure red and fair skin tones look almost identical to a color-based check, because both are 'warm' colors close to the same hue. Fixed by teaching the system to recognize a large block of vivid, glossy color as a painted helmet shell rather than skin -- but only for colors that could plausibly be confused with skin in the first place (a blue shirt was never going to be mistaken for skin, so it shouldn't trigger this check either, which was a second bug found later).

- **The walking pedestrian wrongly flagged as a rider**

Described earlier in this document -- a person merely walking near a parked motorcycle was being linked to it as if they were riding it.

- **A color-correction step that secretly drained all the color**

The shadow-fixing step (Step 1) was implemented in a way that blended in a black-and-white version of the photo, which dulled every color in the image whenever it activated. This broke multiple color-based checks at once. Fixed by correcting only brightness, never color.

- **A bare-headed rider scoring zero**

One rider's photo produced a perfect 0.0 'no helmet' score despite being clearly bare-headed, for two compounding reasons: his shirt collar was mistaken for a helmet (the same color-confusion bug, in a new spot), and his skin and hair

were each genuinely visible but neither alone was quite enough evidence on its own to cross the system's threshold. Fixed by letting partial, corroborating evidence count for something instead of requiring one single signal to be conclusive by itself.

- **A whole tab silently going blank**

A leftover line of code meant 'stop processing this one case' but actually meant 'stop the entire program' -- which made the Video and Dashboard tabs appear completely empty under certain conditions. Found by actually clicking through the live app rather than just reading the code, and fixed by changing how that early exit worked.

The honest conclusion

None of these bugs were found by writing more code and hoping it worked. They were found by deliberately trying to break the system with real, messy, unflattering photos, and being willing to report the failures honestly instead of only showing the cases that worked.

16. Honest Limitations -- What It Can't Do (Yet)

No system like this is perfect, and it's worth being upfront about where this one currently struggles, rather than overselling it.

- **Resolution limits on small or distant riders**

The helmet check relies on examining real pixel detail in someone's head region. If a rider is very small or far away in the photo, there simply isn't enough pixel detail left to analyze reliably, no matter how good the underlying logic is -- this is a hard physical limit, not a fixable bug.

- **Heuristic checks instead of dedicated AI models**

Helmet and seatbelt detection use rule-based color/texture analysis rather than a model specifically trained on thousands of labeled helmet/no-helmet photos. This makes the logic transparent and explainable, but means accuracy depends more on lighting and image quality than a dedicated trained model would.

- **Wrong-side driving on complex roads**

The lane-center estimate assumes a simple two-lane road with visible markings. On wide multi-lane highways, the concept of a single 'lane center' doesn't really apply, so this check is intentionally conservative there.

- **Requires visible road paint**

Stop-line and parking-zone violations are only checked when the system can actually find painted markings in the photo. Faded paint, or roads that never had markings to begin with, mean those specific checks are skipped rather than guessed at.

- **Rain was attempted and deliberately left out**

Two different approaches to spotting rain in a photo were built and tested against a real rainy street photo. Neither could reliably tell the difference between 'this photo has rain' and 'this photo is just a busy, textured street scene' -- so rather than ship a feature that would misfire unpredictably, it was left out and documented honestly instead.

- **Mannequins in shop windows can be mistaken for people**

A storefront mannequin display, photographed at the right angle, can look enough like a real person that the AI model identifies it as one with even higher confidence than it has in some real people. Confidence-based filtering doesn't fix this. A real fix would need to use movement across video frames -- a real person shifts position, a mannequin never does -- which the video mode could support in the future but doesn't yet.

- **Seatbelt and phone-use checks need to see the whole vehicle**

These two checks confirm a person is 'inside a vehicle' by checking if their box overlaps a detected car. That only works for photos taken from outside the vehicle (like a street camera). Photos taken from inside the car looking out -- dashboard-camera style -- never show the car's outside body, so these two checks cannot fire on that kind of photo. This was confirmed by testing directly against real dashboard-style photos, not just assumed.

17. Glossary of Terms Used in This Document

AI model	A program trained on large amounts of example data to recognize patterns -- in this case, recognizing objects in photos.
Bounding box	A rectangle drawn around an object in an image, defined by its corner coordinates, marking exactly where that object is.
Confidence score	A percentage representing how sure the system is about a particular detection or decision.
HSV	A way of describing color using Hue (the actual color, like red or blue), Saturation (how vivid vs. washed-out it is), and Value (how bright vs. dark it is) -- often easier for color-based detection than the standard Red/Green/Blue format.
OCR	Optical Character Recognition -- software that reads text out of an image, used here to read license plate characters.
Heuristic	A practical rule of thumb based on observable patterns (e.g. 'bare skin and rough hair texture usually mean no helmet'), as opposed to a model that learned the pattern from training data.
Hough Transform	A classic image-processing technique for finding straight lines in an image -- used here to confirm stop lines and estimate lane positions.
Pipeline	The fixed sequence of steps a photo goes through from upload to final result -- clean up, detect, link riders, find zones, check violations, read plates, produce evidence.
Precision	Of everything the system flagged, what fraction was actually correct? Low precision means too many false alarms.
Recall	Of everything that was actually a real violation, what fraction did the system catch? Low recall means it's missing real cases.
F1-score	A single number combining precision and recall, so a system can't look good on one while quietly failing the other.
mAP (mean Average Precision)	Another, separate measurement of how well the underlying AI model places its boxes, independent of any violation logic built on top of it.
IoU (Intersection over Union)	A way of measuring how well two boxes overlap -- used to decide whether a predicted box counts as 'correct' against a hand-drawn true box.
Ground truth	The correct answer, written down by a person looking directly at the photo, used to check the system's answer against -- not the system's own output.
False positive / false negative	A false positive is a false alarm (flagged something that wasn't really there); a false negative is a miss (failed to flag something that was).